



Protocol Audit Report

Version 1.0

Cyfrin.io

September 22, 2026

Protocol Audit Report

Cyfrin.io

March 7, 2023

Prepared by: Cyfrin Lead Auditors: - stash21x-crypto

Table of Contents

- Table of Contents
- Protocol Summary
- Disclaimer
- Risk Classification
- Audit Details
 - Scope
 - Roles
- Executive Summary
 - Issues found
- Findings
 - High
 - * [H-#] Erroneous `ThunderLoan::updateExchangeRate` in the `deposit` function causes protocol to think it has more fees than it really does, which blocks redemption and incorrectly sets the exchange rate
 - * [H-#] mixing up variable location causes storage collisions `ThunderLoan::s_flashLoanFee` and `ThunderLoan::s_currentlyFlashLoaning`, freezing protocol
 - Medium
 - * [M-2] Using TSwap as price oracle leads to price and oracle manipulation attacks

Protocol Summary

It lets users borrow capital instantly without collateral as long as they return it within the same transaction, while liquidity providers earn yield from the borrowed capital.

Disclaimer

The YOUR_NAME_HERE team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

		Impact		
		High	Medium	Low
Likelihood	High	H	H/M	M
	Medium	H/M	M	M/L
	Low	M	M/L	L

We use the CodeHawks severity matrix to determine severity. See the documentation for more details.

Audit Details

The findings described in this document correspond the following commit hash:

```
1 2e8f81e263b3a9d18fab4fb5c46805ffc10a9990
```

Scope

```
1  |-- interfaces
2  |    |-- IFlashLoanReceiver.sol
3  |    |-- IPoolFactory.sol
4  |    |-- ITSwapPool.sol
5  |    |-- IThunderLoan.sol
6  |-- protocol
7  |    |-- AssetToken.sol
8  |    |-- OracleUpgradeable.sol
9  |    |-- ThunderLoan.sol
10 |-- upgradedProtocol
11    |-- ThunderLoanUpgraded.sol
```

Roles

Executive Summary

Issues found

Severity	Number of issues found
High	2
Medium	1
Low	0
Info	0
Total	3

Findings

High

[H-#] Erroneous ThunderLoan::updateExchangeRate in the deposit function causes protocol to think it has more fees than it really does, which blocks redemption and incorrectly sets the exchange rate

Description In the ThunderLoan system, the `exchangeRate` is responsible for calculating the exchange rate between assetTokens & underlying tokens. In a way, it's responsible for keeping track of how many fees to give to liquidity providers.

However, the `deposit` function updates this rate, without collecting any fees! This update should be removed.

```
1     function deposit(IERC20 token, uint256 amount) external
2         revertIfZero(amount) revertIfNotAllowedToken(token) {
3         AssetToken assetToken = s_tokenToAssetToken[token];
4         uint256 exchangeRate = assetToken.getExchangeRate();
5         uint256 mintAmount = (amount * assetToken.
6             EXCHANGE_RATE_PRECISION()) / exchangeRate;
7         emit Deposit(msg.sender, token, amount);
8         assetToken.mint(msg.sender, mintAmount);
9         // @audit-high
10        uint256 calculatedFee = getCalculatedFee(token, amount);
11        @> assetToken.updateExchangeRate(calculatedFee);
12
13        // e when a liquidity provider deposits, the $ sits in the
14        assetToken contract
15        token.safeTransferFrom(msg.sender, address(assetToken), amount)
16        ;
17    }
```

Impact There are several impacts to this bug.

1. The `redeem` function is blocked, because the protocol thinks the owed tokens is more than it has
2. Rewards are incorrectly calculated, leading to users potentially getting way more or less than deserved.

Proof Of Concept:

1. LP deposits
2. User takes out a flash loan
3. Now it is impossible for LP to redeem.

Proof Of Code

Place the following into `ThunderLoanTest.t.sol`

```
1     function testRedeemAfterLoan() public setAllowedToken hasDeposits {
2         uint256 amountToBorrow = AMOUNT * 10;
3         uint256 calculatedFee = thunderLoan.getCalculatedFee(tokenA,
4             amountToBorrow);
5
6         vm.startPrank(user);
7         tokenA.mint(address(mockFlashLoanReceiver), calculatedFee); //
8             fee?
9         thunderLoan.flashloan(address(mockFlashLoanReceiver), tokenA,
10             amountToBorrow, "");
11         vm.stopPrank();
12     }
```

```
9
10     uint256 amountToRedeem = type(uint256).max;
11     vm.startPrank(liquidityProvider);
12     thunderLoan.redeem(tokenA, amountToRedeem);
13 }
```

Recommended Mitigation: Removed the incorrectly updated exchange rate lines from `deposit`.

```
1     function deposit(IERC20 token, uint256 amount) external
2         revertIfZero(amount) revertIfNotAllowedToken(token) {
3         AssetToken assetToken = s_tokenToAssetToken[token];
4         uint256 exchangeRate = assetToken.getExchangeRate();
5         uint256 mintAmount = (amount * assetToken.
6             EXCHANGE_RATE_PRECISION()) / exchangeRate;
7         emit Deposit(msg.sender, token, amount);
8         assetToken.mint(msg.sender, mintAmount);
9         // @audit-high
10        - uint256 calculatedFee = getCalculatedFee(token, amount);
11        - assetToken.updateExchangeRate(calculatedFee);
12
13        // e when a liquidity provider deposits, the $ sits in the
14        assetToken contract
15        token.safeTransferFrom(msg.sender, address(assetToken), amount)
16        ;
17    }
```

[H-#] mixing up variable location causes storage collisions ThunderLoan::s_flashLoanFee and ThunderLoan::s_currentlyFlashLoaning, freezing protocol

Description: `ThunderLoan.sol` has two variables in the following order:

```
1     uint256 private s_feePrecision;
2     uint256 private s_flashLoanFee; // 0.3% ETH fee
```

However, the upgraded contract `ThunderLoanUpgraded.sol` has them in a different order:

```
1     uint256 private s_flashLoanFee; // 0.3% ETH fee
2     uint256 public constant FEE_PRECISION = 1e18;
```

Due to how solidity works, after the upgrade the `s_flashLoanFee` will have the value of `s_feePrecision`. You cannot adjust the position of the storage variables, and removing storage variables for constant variables, breaks the storage the storage locations as well.

Impact: After the upgrade, the `s_flashLoanFee` will have the value of `s_feePrecision`. This means that users who take out flash loans right after an upgrade will be charged the wrong fee.

More importantly the `s_currentlyFlashLoaning` mapping with storage in the wrong storage slot.

Proof Of Concept:

PoC

Place the following into `ThunderLoanTest.t.sol`.

```
1 import { ThunderLoanUpgraded } from "../src/upgradedProtocol/  
  ThunderLoanUpgraded.sol";  
2 .  
3 .  
4 .  
5     function testUpgradeBreaks() public {  
6         uint256 feeBeforeUpgrade = thunderLoan.getFee();  
7         vm.startPrank(thunderLoan.owner());  
8         ThunderLoanUpgraded upgraded = new ThunderLoanUpgraded();  
9         thunderLoan.upgradeToAndCall(address(upgraded), "");  
10        uint256 feeAfterUpgrade = thunderLoan.getFee();  
11        vm.stopPrank();  
12  
13        console2.log("Fee Before: ", feeBeforeUpgrade);  
14        console2.log("Fee After: ", feeAfterUpgrade);  
15  
16        assert(feeBeforeUpgrade != feeAfterUpgrade);  
17    }
```

You can also see the storage layout difference by running `forge inspect ThunderLoan storage` and `forge inspect ThunderLoanUpgraded storage`

Recommended Mitigation: If you must remove the storage variable, leave it as blank to not mess up the storage slots.

```
1 -    uint256 private s_flashLoanFee; // 0.3% ETH fee  
2 -    uint256 public constant FEE_PRECISION = 1e18;  
3 +    uint256 private s_blank;  
4 +    uint256 private s_flashLoanFee; // 0.3% ETH fee  
5 +    uint256 public constant FEE_PRECISION = 1e18;
```

Medium**[M-2] Using TSwap as price oracle leads to price and oracle manipulation attacks**

Description: The TSwap protocol is a constant product formula based AMM (automated market maker). The price of a token is determined by how many reserves are on the either side of the pool. Because of this, it is easy for malicious users to manipulate the price of a token by buying or selling a large amount of the token in the same transaction, essentially ignoring protocol fees.

Impact: Liquidity providers will drastically reduce fees for providing liquidity.

Proof Of Concept:

The following all happens in 1 transaction.

1. User takes a flash loan from **ThunderLoan** for 1000 **tokenA**. They are charged the original fee **fee1**. During the flash loan they do the following:
 1. User sells 1000 **tokenA**, tanking the price.
 2. Instead of repaying right away, the user takes out another flash loan for another 1000 **tokenA**.

```
1 function getPriceInWeth(address token) public view returns (
    uint256) {
2     address swapPoolOfToken = IPoolFactory(s_poolFactory).getPool(
        token);
3     @> return ITSwapPool(swapPoolOfToken).
        getPriceOfOnePoolTokenInWeth();
4 }
```

3. The user then repays the first flash loan, and then repays the second flash loan.

Recommended Mitigation: Consider using a different price oracle mechanism, like a Chainlink price feed with a Uniswap TSWAP price fallback oracle.